

INFORME TÉCNICO III

Comunicación, almacenamiento y plataforma web

Integración MQTT, web local, OTA y aplicación cloud VehicleSense

Institución: Escuela Superior Politécnica del Litoral (ESPOL)
Facultad: [FACULTAD O UNIDAD ACADÉMICA]
Carrera: Ingeniería Mecatrónica
Asignatura: [ASIGNATURA / PROYECTO INTEGRADOR]
Autor: Patricio Vásquez ([MATRÍCULA])
Docente: [DOCENTE / TUTOR]
Fecha: Julio de 2026

Resumen ejecutivo

Este informe describe la aplicación integral del proyecto: el ESP32 construye telemetría validada, la respalda en microSD, la publica por MQTT seguro y expone un monitor local para diagnóstico y actualización OTA. En la nube, un broker desacopla el dispositivo de un backend que persiste los mensajes y alimenta un frontend con mapa, vehículos, alertas, viajes, analítica, reportes y administración.

Las capturas cloud corresponden al frontend funcional en **modo demostración**; la propia interfaz advierte que los vehículos y mediciones son simulados. Las capturas locales se generaron cargando el HTML embebido del firmware con respuestas JSON representativas. Por tanto, documentan diseño y funcionalidad, no una conexión física en vivo al ESP32.

Resultado de integración

La solución adopta una ruta principal WiFi + MQTT/TLS. La microSD y una cola offline reducen pérdida de datos, mientras REST/WebSocket separan historia y actualización en tiempo real. El SIM800L queda como transporte experimental; no se desactiva TLS de forma automática si el módem no lo soporta.

Índice general

Resumen ejecutivo	I
1. Arquitectura de extremo a extremo	1
1.1. Componentes	1
1.2. Responsabilidades	1
2. Telemetría y persistencia local	2
2.1. Snapshot y validación	2
2.2. JSON Lines en microSD	2
3. MQTT y conectividad	4
3.1. Sesión segura	4
3.2. Reconexión y entrega	4
3.3. SIM800L y evolución celular	4
4. Web local del ESP32	5
4.1. Propósito	5
4.2. API local	6
4.3. Administración y OTA	6
5. Aplicación cloud VehicleSense	8
5.1. Origen de las capturas	8
5.2. Acceso	8
5.3. Dashboard de flota	9
5.4. Mapa en vivo	10
5.5. Listado y detalle de vehículos	11
5.6. Alertas	13
5.7. Viajes	14
5.8. Analítica	15

5.9. Reportes	16
5.10. Dispositivos	17
5.11. Configuración	18
6. Backend, base de datos y tiempo real	19
6.1. Ingestión	19
6.2. API REST y WebSocket	19
6.3. Tratamiento temporal	19
7. Despliegue y seguridad	20
7.1. Prototipo y producción	20
7.2. Controles mínimos	20
7.3. Limitaciones actuales	20
8. Plan de prueba de integración	21
9. Conclusiones	22

Índice de figuras

1.1. Arquitectura funcional de VehicleSense.	1
4.1. Monitor local. Vista documental del HTML embebido con datos representativos; no es una sesión física del ESP32.	5
4.2. Formulario OTA local. Vista documental; el navegador de captura no representa una actualización ejecutada.	7
5.1. Pantalla de ingreso. En demo permite entrar sin credenciales reales; en producción utiliza JWT emitido por el backend.	8
5.2. Dashboard: indicadores de disponibilidad, mapa resumido, vehículos, tendencias y alertas. Datos simulados.	9
5.3. Mapa en vivo con marcadores y panel de vehículos. Datos simulados.	10
5.4. Listado de vehículos, estado, asignación y resumen de última telemetría. Datos simulados.	11
5.5. Detalle de un vehículo: estado, mapa/ruta, tendencias, telemetría, dispositivo y eventos. Datos simulados.	12
5.6. Bandeja de alertas con severidad, vehículo, instante, ubicación y acciones. Datos simulados.	13
5.7. Resumen de viajes por vehículo con distancia, duración y estado. Datos simulados.	14
5.8. Analítica de tendencias y distribución de variables. Datos simulados.	15
5.9. Exportación de telemetría CSV/JSON y alcance temporal. Datos simulados.	16
5.10. Inventario de dispositivos, firmware, última conexión y vehículo asignado. Datos simulados.	17
5.11. Configuración de sesión, transporte y límites operativos. Modo demostración.	18

Arquitectura de extremo a extremo

1.1 Componentes

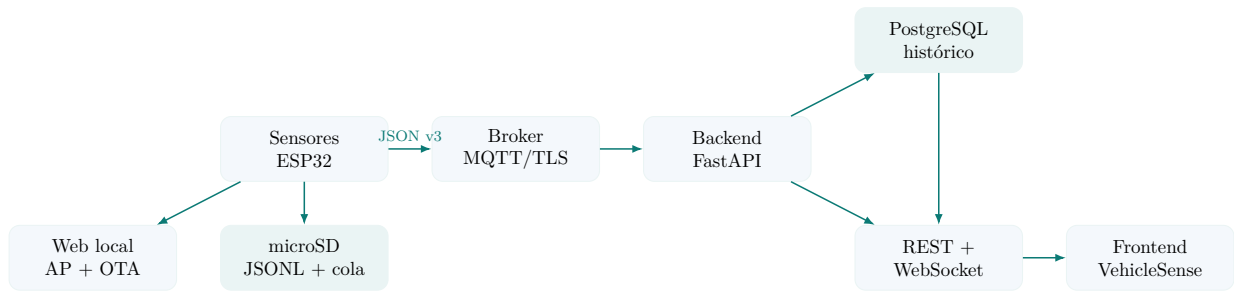


Figura 1.1: Arquitectura funcional de VehicleSense.

1.2 Responsabilidades

Cuadro 1.1: Separación de responsabilidades.

Componente	Responsabilidad	No debe hacer
Firmware	Adquirir, validar, etiquetar, respaldar y publicar	Inventar valores o depender del dashboard.
Broker	Autenticar, autorizar y distribuir mensajes	Ser la base histórica principal.
Backend	Ingerir, normalizar, persistir y servir API	Exponer secretos MQTT al navegador.
Base de datos	Conservar telemetría, eventos y relaciones	Recibir conexiones directas del ESP32.
Frontend	Visualizar, filtrar y gestionar acciones autorizadas	Presentar simulación como medición real.
Web local	Diagnóstico cercano y OTA controlada	Sustituir el portal cloud o guardar gran historia.

Telemetría y persistencia local

2.1 Snapshot y validación

En cada ciclo de publicación, el controlador toma una instantánea de los módulos. La validación considera bandera del driver, finitud, rango y antigüedad. Si el DHT11 falla, se omiten temperatura y humedad pero se conserva `dht_valid=false`; el GPS sin fix no bloquea presión, IMU o luz.

El esquema se versiona para que backend y firmware evolucionen sin ambigüedad. Un mensaje reducido puede ser:

```
{
  "schema_version": 3,
  "device_id": "esp32-devkit-01",
  "vehicle_id": "bus-prototipo-01",
  "sample_id": "17-0000184",
  "uptime_ms": 3720000,
  "time_source": "ntp",
  "temperature_c": 27.4,
  "humidity_percent": 58.0,
  "light_lux": 8240.0,
  "pressure_hpa": 1006.72,
  "baro_altitude_m": 10.4,
  "dht_valid": true,
  "gps_valid": false,
  "imu_valid": true,
  "sd_valid": true
}
```

Los ejes `accel_x/y/z` son valores calibrados; los raw quedan en diagnóstico. La presión publicada es local, mientras `sea_level_pressure_hpa` describe la referencia usada para la altitud barométrica. Nunca se serializa NaN, null ni un cero ficticio.

2.2 JSON Lines en microSD

Cada muestra aceptada se escribe primero como una línea JSON en un archivo por sesión y segmento. Este formato facilita recuperación parcial y procesamiento por lotes. La escritura se confirma antes de marcar el estado como saludable. Los archivos rotan por tamaño y no se eliminan automáticamente en la fase académica.

Una cola offline acotada registra mensajes no confirmados. Al recuperar MQTT, el dispositivo puede reenviar respetando identificadores de muestra para permitir deduplicación. Deben reportarse

profundidad, bytes, muestras reproducidas y descartes.

Precaución

microSD no equivale a una base transaccional. Un corte durante escritura puede dañar la última línea o el sistema FAT. La campaña necesita extracción controlada, capacidad monitorizada y copia frecuente. Sin RTC/NTP, el archivo usa sesión y uptime, no una fecha inventada.

MQTT y conectividad

3.1 Sesión segura

MQTT utiliza el patrón publish/subscribe y permite desacoplar productores y consumidores [1]. La ruta cloud propuesta exige TLS, credencial independiente por dispositivo y ACL limitada a sus temas. El cliente publica con QoS 1, conserva el token hasta PUBACK y utiliza LWT para exponer desconexión inesperada.

Cuadro 3.1: Familia de temas propuesta.

Tema	Uso
vehicles/<id>/telemetry	Muestras completas o parciales.
vehicles/<id>/status	Online, versión, uptime y salud.
vehicles/<id>/events	Eventos derivados con severidad y contexto.
vehicles/<id>/commands	Comandos autenticados; requiere diseño de respuesta.
vehicles/<id>/ack	Confirmación de comandos y correlación.

3.2 Reconexión y entrega

WiFi y MQTT poseen backoff independiente. La adquisición nunca espera de forma indefinida una conexión. Si WiFi cae, el dispositivo sigue escribiendo en microSD; al volver, MQTT restablece sesión y la cola inicia replay. El backend deduplica por `device_id+sample_id`.

3.3 SIM800L y evolución celular

La interfaz de MQTT recibe un `Client`, por lo que puede operar sobre WiFi, TCP celular o TLS celular. Sin embargo, SIM800L depende de 2G y su soporte SSL varía. La secuencia experimental es AT → SIM → registro → APN/GPRS → TCP sintético → TLS/SNI → MQTT. Si TLS falla, el perfil integrado conserva datos localmente y recomienda un módem LTE; no transmite telemetría real en texto plano por defecto.

Web local del ESP32

4.1 Propósito

El ESP32 puede mantener un Access Point WPA2 y atender HTTP sin recursos externos. El dashboard consulta `/api/status` y `/api/telemetry/basic` cada dos segundos. Muestra variables ambientales, magnitudes resumidas de IMU, almacenamiento, conectividad, MQTT, audio, uptime y alertas. La interfaz permanece útil aunque Internet o el broker no estén disponibles.

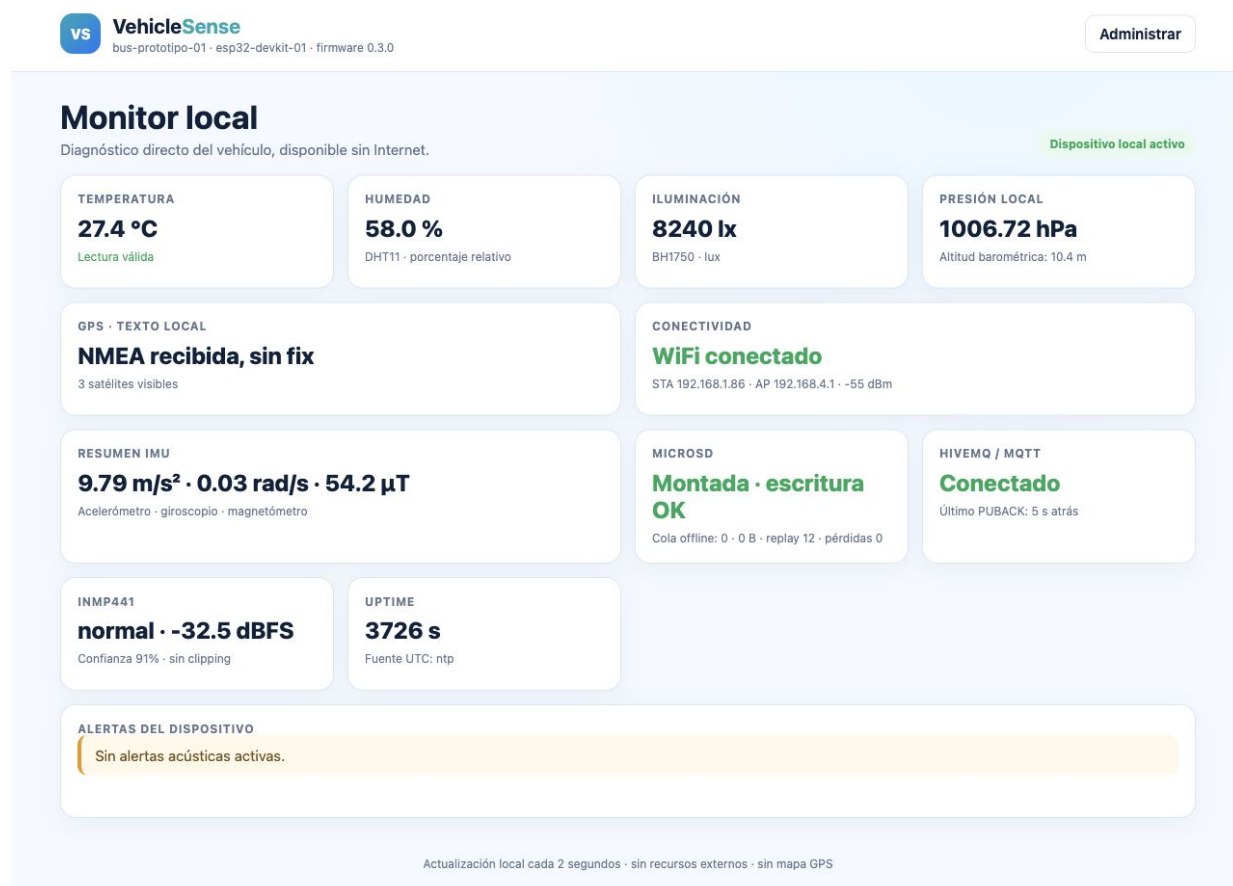


Figura 4.1: Monitor local. Vista documental del HTML embebido con datos representativos; no es una sesión física del ESP32.

La figura muestra un estado saludable, GPS con flujo NMEA pero sin fix, microSD montada y MQTT conectado. Los valores se escogieron únicamente para recorrer los estados visuales. El pie

de página comunica actualización local y ausencia de mapa.

4.2 API local

Cuadro 4.1: Endpoints HTTP embebidos.

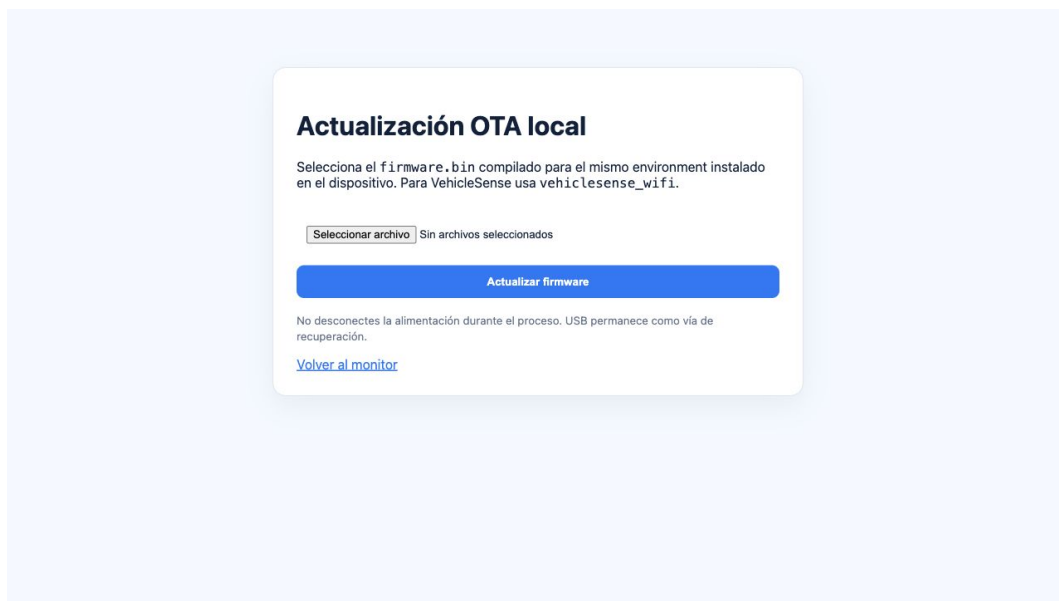
Método	Ruta	Función
GET	/	Dashboard responsive de diagnóstico.
GET	/api/status	Red, SD, MQTT, cola, OTA, sesión y uptime.
GET	/api/telemetry/basic	Sensores básicos y magnitudes resumidas.
GET	/admin	Formulario de actualización autenticado.
POST	/admin/update	Recibe <code>firmware.bin</code> y ejecuta Update.

Precaución

Hallazgo de revisión: aunque el requisito original indicaba no mostrar GPS en la web local, el HTML actual contiene una tarjeta GPS y la API básica puede incluir latitud y longitud cuando el fix es válido. La vista capturada no expone coordenadas porque el ejemplo usa `gps_valid=false`, pero el contrato sí las contempla. Debe decidirse si se elimina esa información antes de una prueba con usuarios.

4.3 Administración y OTA

`/admin` exige autenticación HTTP Basic adicional. El usuario selecciona el `firmware.bin` generado para el mismo environment. El firmware valida cada bloque, llama a `Update.end(true)` y reinicia solo después de responder éxito. La tabla de particiones conserva dos slots OTA; USB permanece como recuperación.



The image shows a web form titled "Actualización OTA local" (Local OTA Update). The form is centered on a light blue background. It contains the following elements:

- Title:** "Actualización OTA local" in bold black text.
- Instructions:** "Selecciona el firmware .bin compilado para el mismo environment instalado en el dispositivo. Para VehicleSense usa vehiclesense_wifi."
- File Selection:** A button labeled "Seleccionar archivo" followed by the text "Sin archivos seleccionados".
- Update Button:** A large blue button labeled "Actualizar firmware".
- Warning:** "No desconectes la alimentación durante el proceso. USB permanece como vía de recuperación."
- Link:** A blue link labeled "Volver al monitor".

Figura 4.2: Formulario OTA local. Vista documental; el navegador de captura no representa una actualización ejecutada.

La autenticación Basic depende de WPA2 y no sustituye firma criptográfica. Una versión posterior debe añadir firma, política de downgrade y rollback confirmado.

Aplicación cloud VehicleSense

5.1 Origen de las capturas

El frontend se ejecutó localmente en modo demostración. Una franja amarilla identifica que vehículos y mediciones son simulados. Las capturas sirven para documentar la arquitectura de información y los flujos de usuario; no prueban ingestión desde un bus.

5.2 Acceso

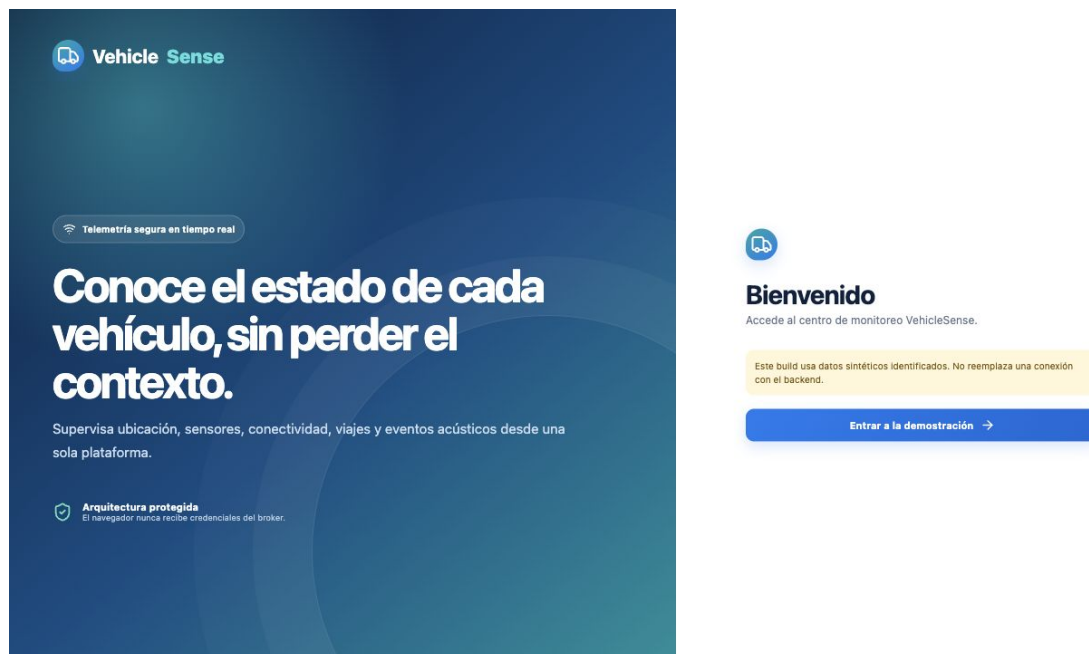


Figura 5.1: Pantalla de ingreso. En demo permite entrar sin credenciales reales; en producción utiliza JWT emitido por el backend.

El acceso separa autenticación de navegación. En producción, el backend aplica roles, duración de token y auditoría; el navegador nunca recibe credenciales MQTT de escritura.

5.3 Dashboard de flota

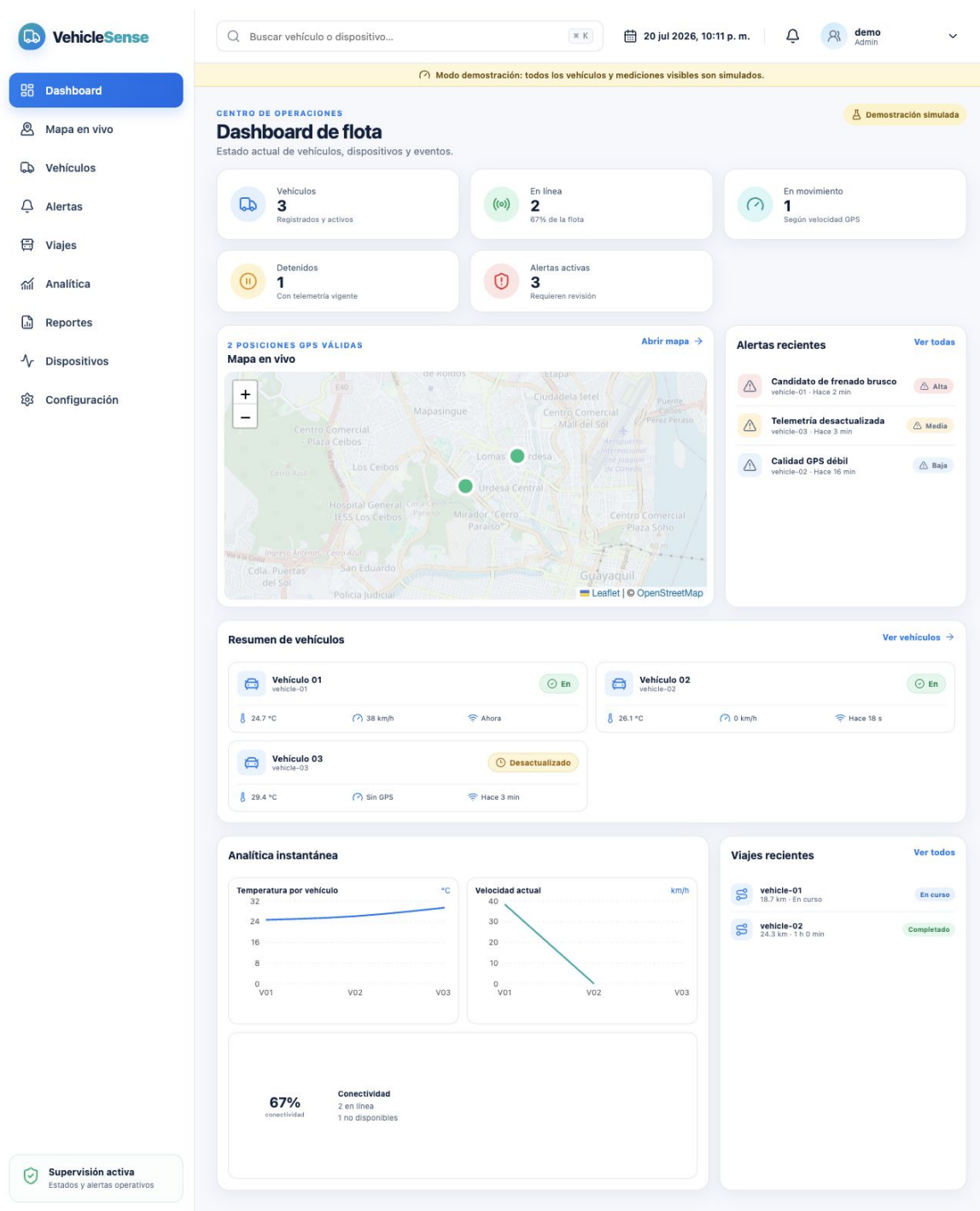


Figura 5.2: Dashboard: indicadores de disponibilidad, mapa resumido, vehículos, tendencias y alertas. Datos simulados.

Su propósito es responder rápidamente cuántos vehículos están activos, cuál requiere atención y dónde se concentra la actividad. Las tarjetas deben definirse con reglas operativas explícitas: “online” por última muestra, “en movimiento” por velocidad y “alerta” por evento no resuelto.

5.4 Mapa en vivo

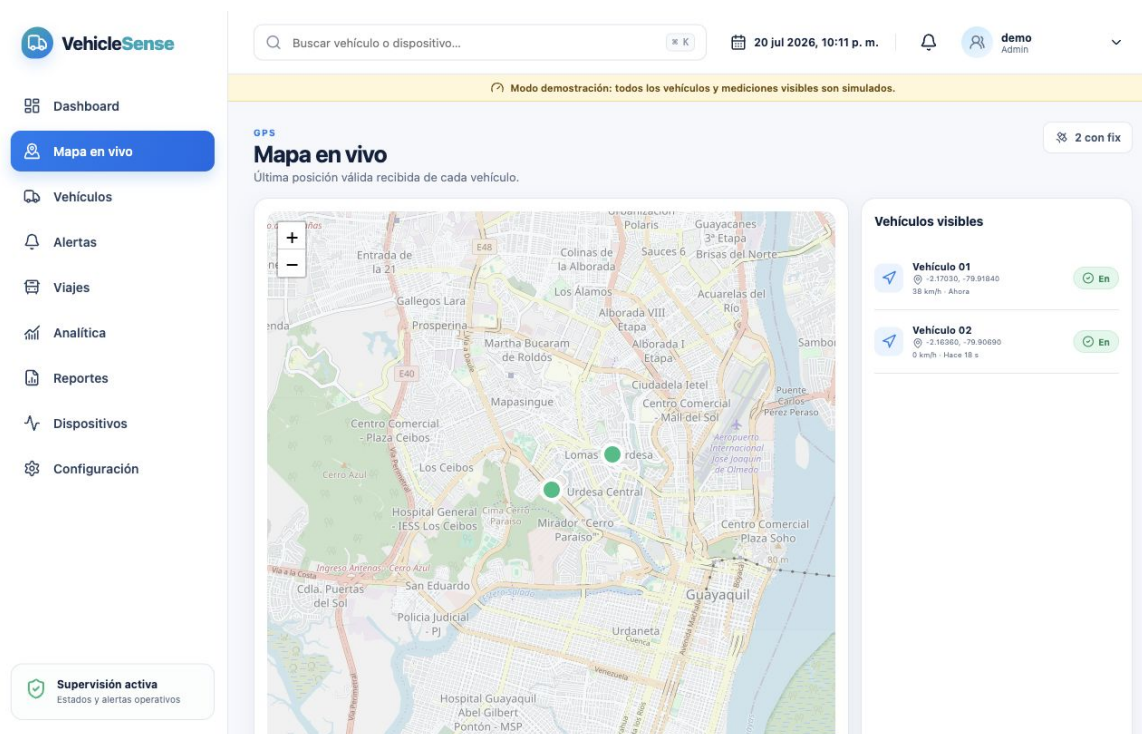


Figura 5.3: Mapa en vivo con marcadores y panel de vehículos. Datos simulados.

La página permite seleccionar un vehículo y centrar su posición. La nube sí puede mostrar GPS porque existe autenticación y control de roles; se debe limitar precisión y retención según la política institucional.

5.5 Listado y detalle de vehículos

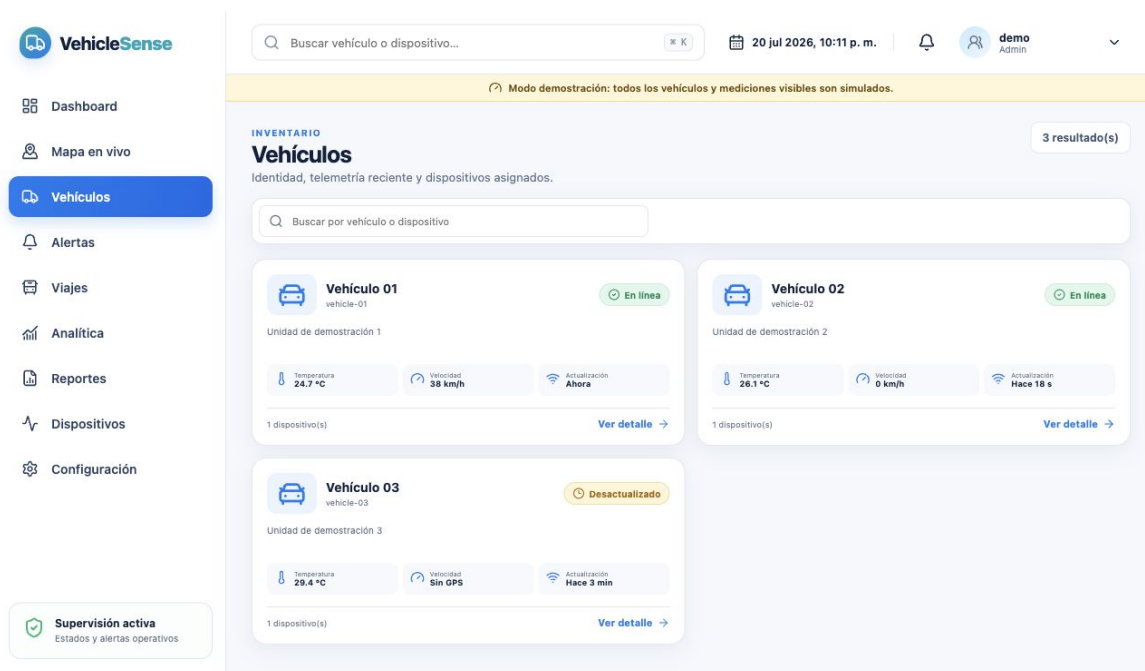


Figura 5.4: Listado de vehículos, estado, asignación y resumen de última telemetría. Datos simulados.

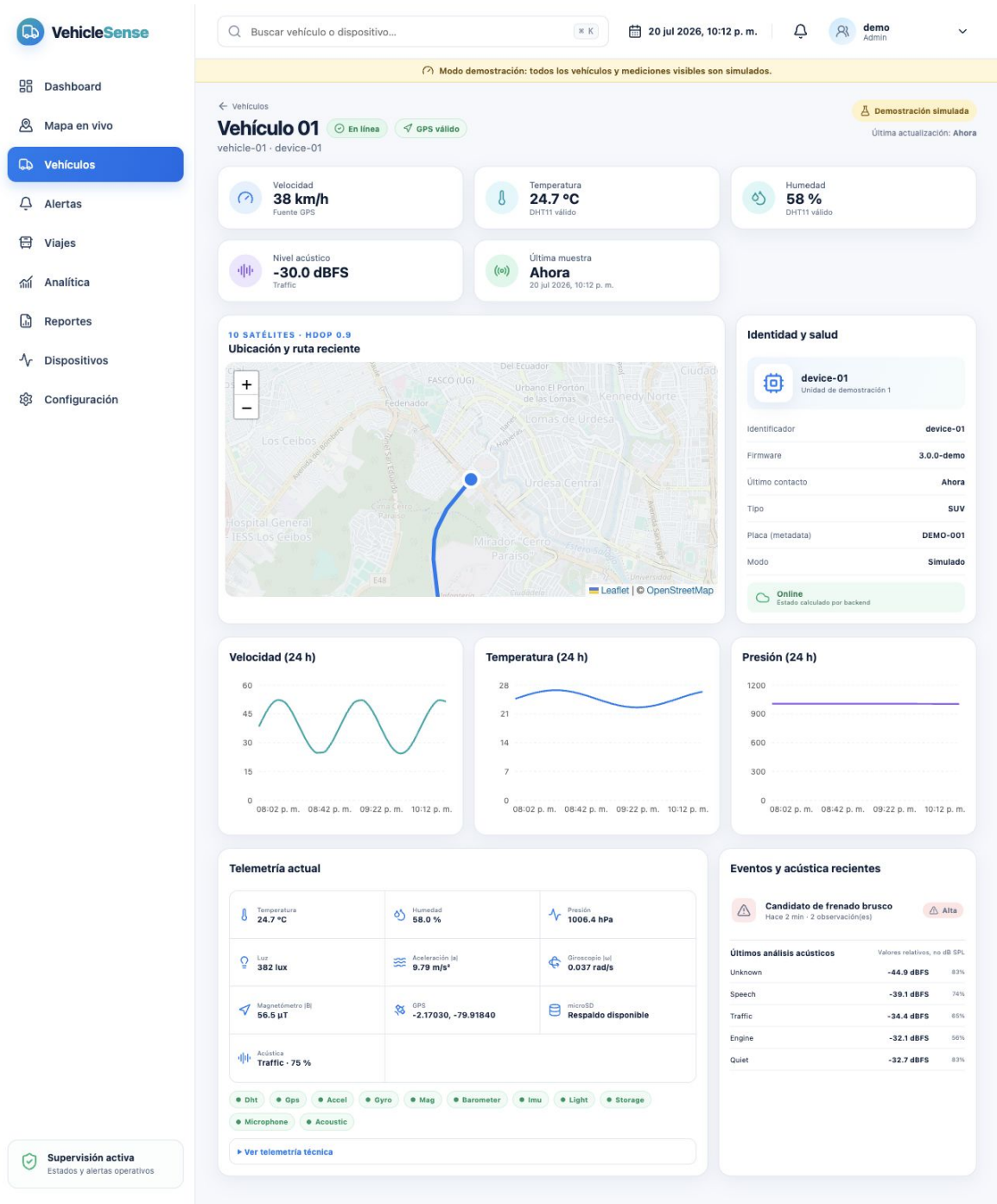


Figura 5.5: Detalle de un vehículo: estado, mapa/ruta, tendencias, telemetría, dispositivo y eventos. Datos simulados.

El detalle consolida las variables de un solo bus. Las gráficas distinguen muestras históricas y actualización en vivo; la sección de salud muestra firmware, conectividad y microSD. La validez del sensor debe acompañar cada valor y no ocultarse con un cero.

5.6 Alertas

VehicleSense

Dashboard
Mapa en vivo
Vehículos
Alertas
Viajes
Analítica
Reportes
Dispositivos
Configuración

Supervisión activa
Estados y alertas operativos

Buscar vehículo o dispositivo... 20 jul 2026, 10:12 p. m. demo Admin

Modo demostración: todos los vehículos y mediciones visibles son simulados.

EVENTOS
Alertas
Revisa, reconoce y resuelve condiciones detectadas.

Estado: Todas Activa Reconocida Resuelta

ALERTA	VEHÍCULO	SEVERIDAD	ESTADO	ÚLTIMA OBSERVACIÓN	ACCIONES
Candidato de frenado brusco Aceleración o frenado brusco - 2 observación(es)	vehicle-01	Alta	Activa	20 jul 2026, 10:10 p. m.	Reconocer Resolver
Telemetría desactualizada Telemetría desactualizada - 1 observación(es)	vehicle-03	Media	Activa	20 jul 2026, 10:08 p. m.	Reconocer Resolver
Calidad GPS débil Calidad GPS débil - 4 observación(es)	vehicle-02	Baja	Reconocida	20 jul 2026, 9:57 p. m.	Resolver

Figura 5.6: Bandeja de alertas con severidad, vehículo, instante, ubicación y acciones. Datos simulados.

Las alertas requieren estado (nueva, reconocida, resuelta), responsable, comentario y trazabilidad. Un umbral no debe convertirse en alerta hasta definir ventana, histeresis, calidad del sensor y severidad. El frontend permite filtrar y reconocer; el backend conserva el cambio.

5.7 Viajes

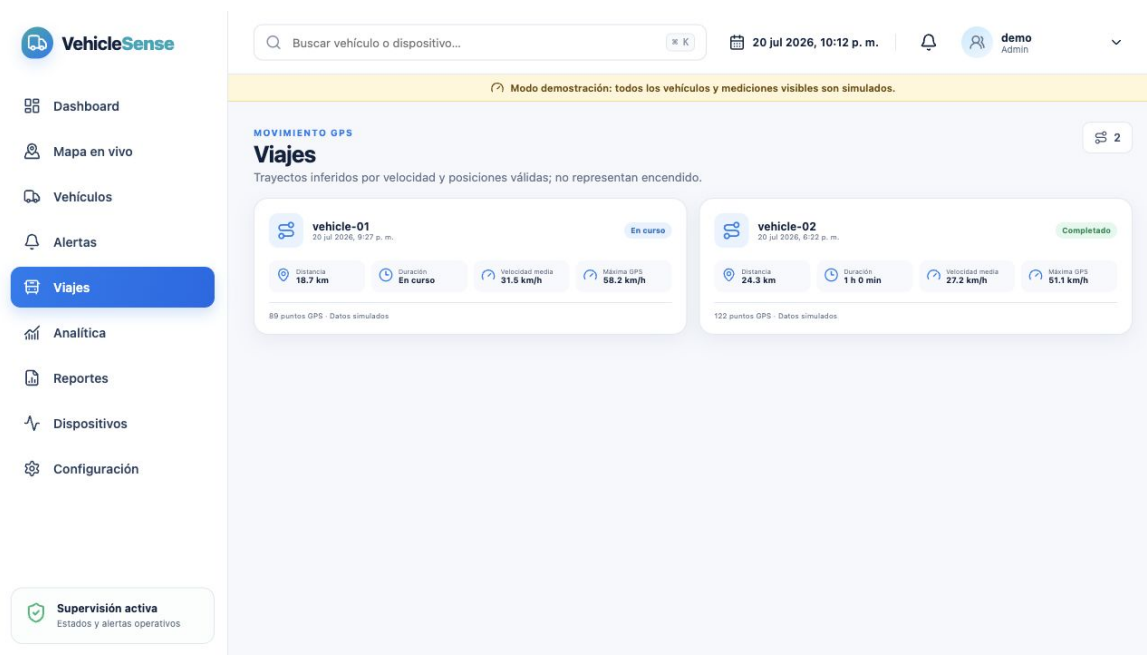


Figura 5.7: Resumen de viajes por vehículo con distancia, duración y estado. Datos simulados.

Un viaje se deriva de posición, velocidad y reglas de inicio/fin. La ausencia de GPS debe producir un viaje incompleto, no una ruta interpolada presentada como exacta.

5.8 Analítica

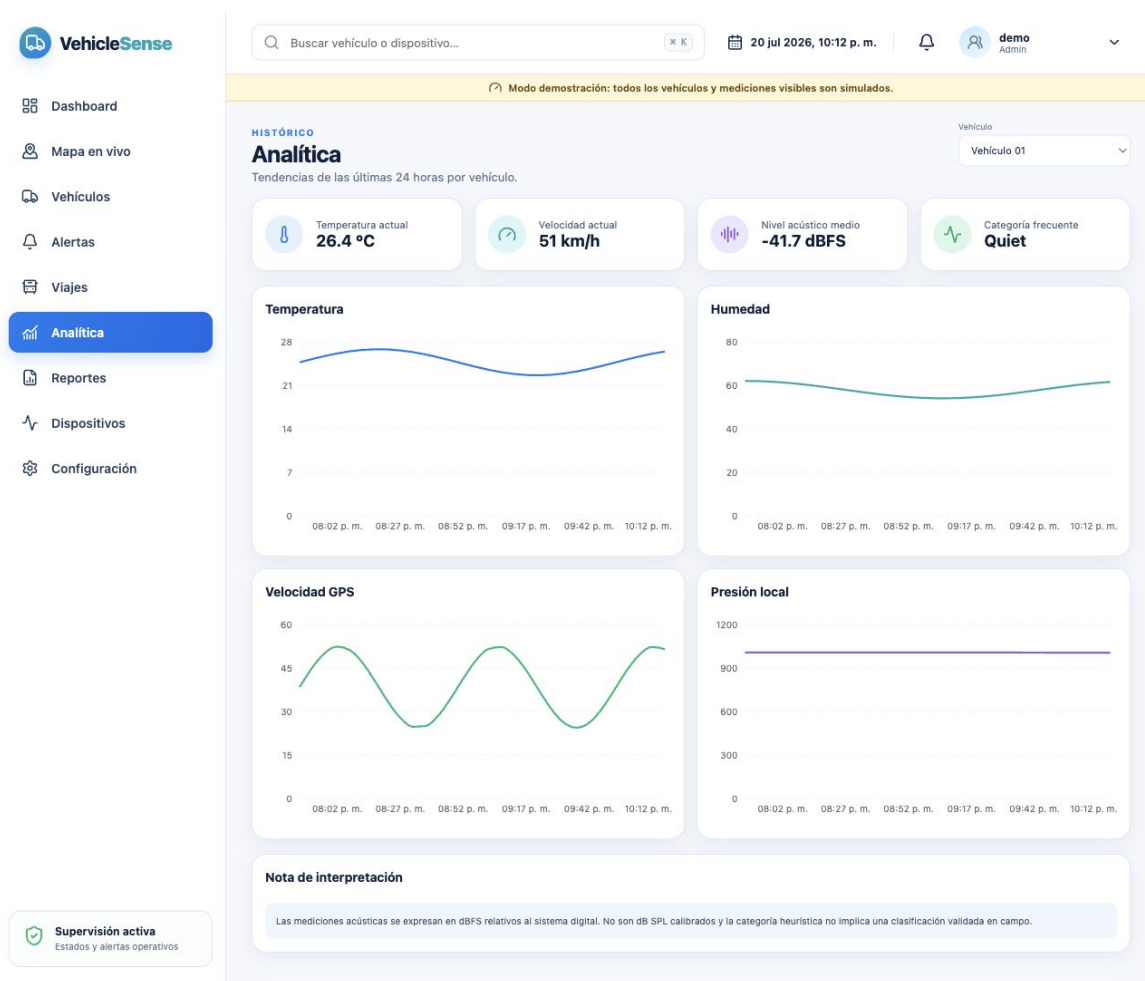


Figura 5.8: Analítica de tendencias y distribución de variables. Datos simulados.

La página compara temperatura, humedad, velocidad y otras métricas en una ventana seleccionada. La interpretación debe mantener unidad, número de muestras, zona horaria y huecos de conectividad. Los datos simulados permiten validar el diseño, no conclusiones sobre la flota real.

5.9 Reportes

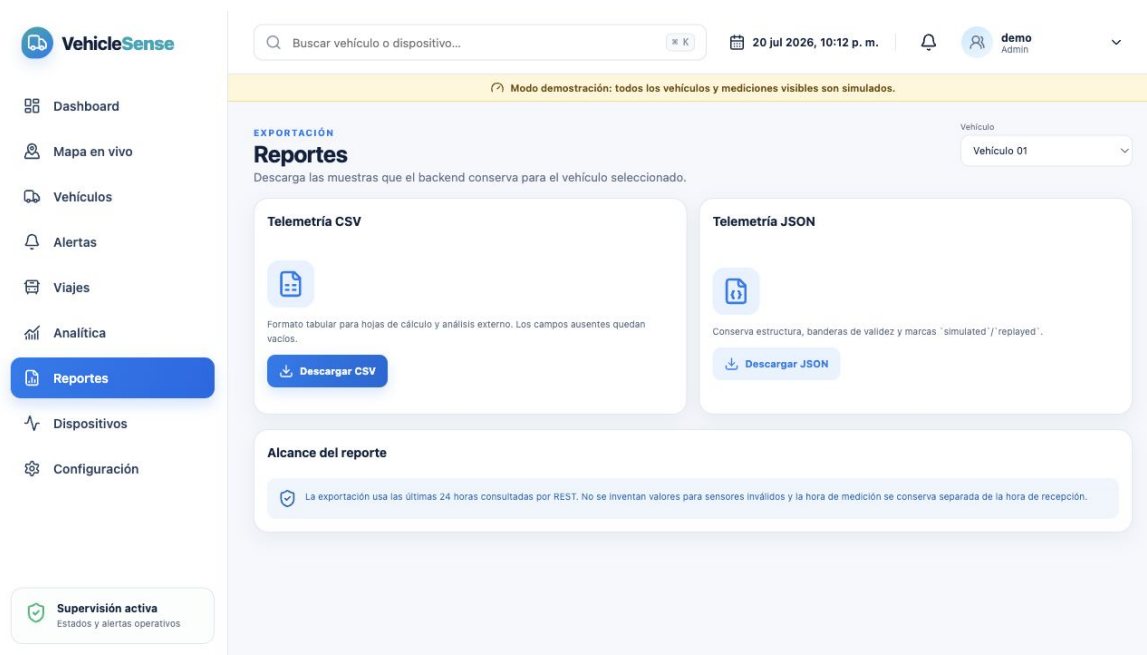


Figura 5.9: Exportación de telemetría CSV/JSON y alcance temporal. Datos simulados.

CSV facilita hojas de cálculo y JSON conserva estructura/flags. Toda exportación debe registrar filtros, zona horaria, esquema y vehículo; para grandes periodos conviene un trabajo asíncrono con enlace de descarga temporal.

5.10 Dispositivos

VehicleSense

Buscar vehículo o dispositivo... 20 jul 2026, 10:12 p. m. demo Admin

Modo demostración: todos los vehículos y mediciones visibles son simulados.

HARDWARE

Dispositivos
Inventario, firmware y conectividad calculada.

device-01
device-01

En línea

Vehículo —
Firmware 3.0.0-demo
Último contacto Ahora
Origen Simulado
Sin razón reportada

device-02
device-02

En línea

Vehículo —
Firmware 3.0.0-demo
Último contacto Hace 18 s
Origen Simulado
Sin razón reportada

device-03
device-03

Desactualizado

Vehículo —
Firmware 3.0.0-demo
Último contacto Hace 3 min
Origen Simulado
Sin razón reportada

Supervisión activa
Estados y alertas operativos

Figura 5.10: Inventario de dispositivos, firmware, última conexión y vehículo asignado. Datos simulados.

Esta pantalla separa identidad electrónica de identidad vehicular. Permite reemplazar un ESP32 sin alterar el historial del bus, además de observar versión, estado y última actividad. Las operaciones sensibles necesitan rol administrador.

5.11 Configuración

The screenshot displays the 'Configuración' (Configuration) page within the VehicleSense application. The interface includes a sidebar with navigation options: Dashboard, Mapa en vivo, Vehículos, Alertas, Viajes, Analítica, Reportes, and Dispositivos. The 'Configuración' option is highlighted. The main content area is titled 'ADMINISTRACIÓN Configuración' and includes a subtitle 'Resumen de sesión, transporte y límites operativos.' A yellow banner at the top indicates 'Modo demostración: todos los vehículos y mediciones visibles son simulados.' The configuration is divided into four sections:

- Sesión actual:** Displays user information: Usuario (demo@vehiclesense.local), Rol (admin), and Autenticación (JWT access + refresh).
- Fuente de datos:** Lists data sources: Modo (Demostración simulada), Historial (REST /api/v1), and Datos en vivo (WebSocket FastAPI).
- Configuración operativa:** Contains a note: 'Los umbrales por vehículo, usuarios y asignaciones se gestionan mediante endpoints protegidos del backend. Este primer cliente muestra el estado efectivo sin exponer credenciales HiveMQ ni secretos de infraestructura en el navegador.'
- Seguridad:** Lists security notes:
 - HiveMQ solo es accesible desde dispositivos y backend mediante ACL específicas.
 - PostgreSQL no se expone al navegador.
 - Los tokens se conservan en 'sessionStorage' y se eliminan al cerrar sesión.
 - Producción debe servirse únicamente por HTTPS detrás de Nginx.

At the bottom left, there is a 'Supervisión activa' (Active Monitoring) status indicator showing 'Estados y alertas operativos'.

Figura 5.11: Configuración de sesión, transporte y límites operativos. Modo demostración.

La configuración centraliza perfil del usuario y parámetros permitidos. Los secretos no se muestran ni se almacenan en el frontend. Cualquier cambio de límites debe quedar versionado y auditado.

Backend, base de datos y tiempo real

6.1 Ingestión

Un consumidor MQTT persistente valida autenticidad, tema y esquema antes de insertar. Las tablas separan dispositivos, vehículos, telemetría, viajes, alertas y eventos de auditoría. Un índice por vehículo e instante soporta gráficas; una restricción única por identificador de muestra evita duplicados del replay.

6.2 API REST y WebSocket

REST atiende consultas históricas, inventario, alertas y exportaciones. WebSocket distribuye la última muestra a clientes autorizados. El backend no confía en el rol enviado por el navegador: lo deriva del token y aplica autorización por endpoint.

6.3 Tratamiento temporal

El dispositivo declara fuente de tiempo: NTP, GPS o none. El backend conserva tiempo de muestra y tiempo de recepción. Si solo existe uptime, no convierte arbitrariamente a UTC. Los dashboards deben indicar desactualización y zona horaria.

Despliegue y seguridad

7.1 Prototipo y producción

Para prototipo, el stack puede ejecutarse con contenedores: broker, backend, PostgreSQL, frontend y proxy inverso. Un servidor persistente es preferible a funciones que se duermen para el consumidor MQTT. Vercel puede alojar el frontend estático, pero la ingestión histórica requiere un proceso siempre activo.

7.2 Controles mínimos

- TLS para MQTT y HTTPS para API/web.
- ACL por dispositivo y credenciales rotantes.
- JWT corto, refresh controlado y roles backend.
- Secretos en variables de entorno o NVS, nunca en Git ni JavaScript público.
- Rate limit, validación de tamaño JSON y límites de consulta/exportación.
- Copias de seguridad, logs sin contraseñas y auditoría de acciones.
- Retención y minimización de GPS; no conservar audio crudo.

7.3 Limitaciones actuales

- Las capturas cloud usan datos demo; falta campaña con dispositivos reales.
- El dashboard local capturado usa JSON representativo.
- SIM800L no garantiza TLS/SNI estable y 2G puede no estar disponible.
- La cola offline es acotada; un corte prolongado puede agotar espacio.
- Basic Auth OTA no proporciona firma de firmware.
- GPS local debe revisarse contra el requisito de privacidad original.
- Las alertas de movimiento y audio requieren validación de campo.

Plan de prueba de integración

Bloque	Prueba	Aceptación
Payload	Sensor inválido, NaN, todos inválidos, buffer límite	JSON parseable; flags presentes; números inválidos omitidos.
microSD	60 muestras, retiro, reinserción, reinicio	Archivo JSONL legible; loop continúa; estado refleja fallo.
MQTT	TLS, ACL, QoS 1, broker reiniciado	PUBACK observado; reconexión; sin texto plano involuntario.
Offline	Cortar red 10 min y restablecer	Cola crece, replay se confirma y no duplica en backend.
Web local	AP sin Internet, APIs, móvil, sensor ausente	Interfaz responsive, sin bloqueo y estados honestos.
OTA	Credencial incorrecta, bin inválido, bin válido	401/rechazo; reinicio solo en éxito; USB recupera.
Backend	Duplicado, esquema inválido, usuario sin rol	Deduplicación, 4xx y autorización correcta.
Cloud	REST, WebSocket, filtros, exportación	Datos coherentes, zona horaria y modo demo visibles.
Soak	Operación 60 min con fallos inducidos	Sin watchdog; sensores y respaldo continúan.
Ruta	Recorrido controlado con referencia	Posición, viaje y eventos comparables con evidencia externa.

Conclusiones

La aplicación completa convierte el prototipo sensorial en un sistema observable: el dispositivo conserva datos cuando falla la red, MQTT desacopla la ingestión y la plataforma presenta estado operativo e histórico. La web local resuelve diagnóstico y OTA sin depender de Internet; la web cloud concentra GPS, rutas, gráficas, alertas y administración.

El diseño visual está avanzado y el software cuenta con modo demostración, pero aún debe conectarse a una campaña física documentada. La siguiente evidencia clave es una integración de 60 minutos, corte/recuperación de red, replay desde microSD y recorrido real. También debe resolverse la exposición de GPS en la API local para que la implementación coincida con el requisito de privacidad.

Bibliografía

- [1] OASIS. *MQTT Version 3.1.1, OASIS Standard*. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- [2] PlatformIO. *Espressif 32: partition tables and build options*. <https://docs.platformio.org/en/latest/platforms/espressif32.html>
- [3] FastAPI. *Official documentation*. <https://fastapi.tiangolo.com/>
- [4] PostgreSQL Global Development Group. *PostgreSQL documentation*. <https://www.postgresql.org/docs/>